

```

import os, re, logging, aiohttp, asyncio
from motor.motor_asyncio import
AsyncIOMotorClient
from pymongo.errors import
DuplicateKeyError
from html import escape as he
from datetime import datetime
from dotenv import load_dotenv

load_dotenv()

from telegram import Update,
InlineKeyboardButton, InlineKeyboardMarkup
from telegram.ext import (
    Application, CommandHandler,
    MessageHandler,
    CallbackQueryHandler, ContextTypes,
    filters,
)
from telegram.constants import ParseMode
from telegram.error import BadRequest

#
=====

=====

# ⚙️ CONFIG
#
=====

=====

BOT_TOKEN =

```

```
"8794125671:AAE2Nq_UXRU0XYvVVjhclmvQA1xdhar7jGw"
```

```
OWNER_ID      = 7302427268
```

```
OWNER_USERNAME = "l_Smoke_ll"
```

```
MONGO_URI      =
```

```
"mongodb+srv://yb131567_db_user:R8zxuvc9Qn999Arg@cluster0.drjaxl8.mongodb.net/telegram_bot?retryWrites=true&w=majority"
```

```
# — APIs —
```

```
TG2PHONE_API_URL = "https://project-fawn-eight-95.vercel.app/tg2phone/api"
```

```
TG2PHONE_API_KEY = "yadav"
```

```
PHONE_API_URL     =
```

```
"https://nmdllpezcocquamhgpmc.supabase.co/functions/v1/lookup"
```

```
# — Daily Limit —
```

```
DAILY_LIMIT = 10
```

```
logging.basicConfig(format="%(asctime)s | %(levelname)s | %(message)s",
```

```
level=logging.INFO)
```

```
logger = logging.getLogger(__name__)
```

```
HTML = ParseMode.HTML
```

```
#
```

```
#  SAFE HELPERS
```

```
#
```

```

def strip_html(text: str) -> str:
    return re.sub(r"<[>]+>", "", text)

async def safe_send(fn, text: str,
reply_markup=None, parse_mode=HTML):
    try:
        return await fn(text,
parse_mode=parse_mode,
reply_markup=reply_markup)
    except BadRequest as e:
        if any(x in str(e).lower() for x in
("can't parse", "bad request", "invalid",
"entities")):
            logger.warning("HTML rejected,
retrying plain: %s", e)
            try:
                return await
fn(strip_html(text), parse_mode=None,
reply_markup=reply_markup)
            except Exception as e2:
                logger.error("Plain
fallback failed: %s", e2)
        else:
            raise

async def safe_edit(msg, text: str,
reply_markup=None, parse_mode=HTML):
    try:
        await msg.edit_text(text,
parse_mode=parse_mode,
reply_markup=reply_markup)
    except BadRequest as e:
        err = str(e).lower()
        if any(x in err for x in ("can't

```

```

parse", "bad request", "invalid",
"entities")):
    logger.warning("HTML rejected
in edit, retrying plain: %s", e)
    try:
        await
msg.edit_text(strip_html(text),
parse_mode=None, reply_markup=reply_markup)
    except Exception as e2:
        logger.error("Plain edit
fallback failed: %s", e2)
    elif "message to edit not found" in
err:
        try:
            await msg.reply_text(text,
parse_mode=parse_mode,
reply_markup=reply_markup)
        except Exception as e2:
            logger.error("safe_edit
reply fallback: %s", e2)
    elif "message is not modified" in
err:
        pass
    else:
        logger.error("safe_edit error:
%s", e)

```

```

#

```

```

# DATABASE

```

```

#

```

```

_mongo_client = None
_mdb           = None

async def init_db():
    global _mongo_client, _mdb
    _mongo_client =
    AsyncIOMotorClient(MONGO_URI)
    _mdb =
    _mongo_client.get_default_database()

    try:
        pipeline = [
            {"$group": {"_id": "$user_id",
"count": {"$sum": 1}, "ids": {"$push":
"$_id"}}},
            {"$match": {"count": {"$gt":
1}}}
        ]
        duplicates = await
        _mdb.users.aggregate(pipeline).to_list(leng
th=None)
        for dup in duplicates:
            remove_ids = dup["ids"][1:]
            await
            _mdb.users.delete_many({"_id": {"$in":
remove_ids}})
        except Exception as e:
            logger.warning("Dedup step failed
(maybe collection empty?): %s", e)

    try:
        await
        _mdb.users.create_index("user_id",

```

```

unique=True)
    except DuplicateKeyError:
        await
_mdb.users.create_index("user_id")

    logger.info("✅ MongoDB connected")

def _default_user(uid, username="",
full_name="", language_code=""):
    now = datetime.now().strftime("%Y-%m-%d
%H:%M:%S")
    today = datetime.now().strftime("%Y-%m-
%d")
    return {
        "user_id": uid,
        "username": username,
        "full_name": full_name,
        "language_code": language_code,
        "last_seen": now,
        "joined_at": now,
        "daily_lookups": 0,
        "last_reset_date": today,
    }

async def upsert_user(u):
    now = datetime.now().strftime("%Y-%m-%d
%H:%M:%S")
    default = _default_user(u.id,
u.username or "", u.full_name or "",
u.language_code or "")
    set_fields = {
        "username": u.username or "",
        "full_name": u.full_name or "",
        "language_code": u.language_code or

```

```

    """
        "last_seen": now,
    }
    on_insert = {k: v for k, v in
default.items() if k not in set_fields}
    await _mdb.users.update_one(
        {"user_id": u.id},
        {"$set": set_fields,
"$setOnInsert": on_insert},
        upsert=True,
    )

async def get_user(user_id):
    return await
_mdb.users.find_one({"user_id": user_id},
{"_id": 0})

async def get_all_users():
    cursor = _mdb.users.find({}, {"_id":
0}).sort("joined_at", -1)
    return await
cursor.to_list(length=None)

def is_owner(user_id):
    return user_id == OWNER_ID

#
=====
=====
# 🗝️ DAILY LIMIT (fixed at 10, no
bonuses)
#
=====

```

```

async def
check_and_increment_lookup(user_id: int):
    today = datetime.now().strftime("%Y-%m-%d")
    user = await get_user(user_id)
    if not user:
        return False, "User not found"

    if user.get("last_reset_date") !=
today:
        await _mdb.users.update_one(
            {"user_id": user_id},
            {"$set": {"daily_lookups": 0,
"last_reset_date": today}}
        )
        user["daily_lookups"] = 0

    daily = user.get("daily_lookups", 0)
    if daily >= DAILY_LIMIT:
        return False, (
            f"❌ <b>Daily limit reached!
</b>\n\n"
            f"📊 You used <code>{daily}</code>
out of <code>{DAILY_LIMIT}</code>
lookups today.\n"
            f"🕒 Limit resets at midnight."
        )

    await _mdb.users.update_one({"user_id":
user_id}, {"$inc": {"daily_lookups": 1}})
    return True, None

```

```

#
=====

# 🌐 API CALLS
#
=====

async def fetch_info(query: str) -> dict:
    try:
        timeout =
aiohttp.ClientTimeout(total=15, connect=5)
        async with aiohttp.ClientSession()
as s:
            async with
s.get(TG2PHONE_API_URL, params={"key":
TG2PHONE_API_KEY, "q": query},
timeout=timeout) as r:
                if r.status != 200:
                    return {"success":
False, "message": f"API Error {r.status}"}
                    data = await
r.json(content_type=None)

                    if not data or not
data.get("result", {}).get("success"):
                        return {"success":
False, "message": "Not found in API"}

                    result = data.get("result",
{})
                    record =
result.get("record", {})

                    name = "-"

```

```

        name_history =
record.get("name_history", [])
        if name_history and
instance(name_history, list):
            latest =
name_history[-1]
            names =
latest.get("usernames_and_names", [])
            if names:
                for n in names:
                    if not
n.startswith("@"):
                                name = n
                                break

        return {
            "success": True,
            "user_id":
result.get("tg_id"),
            "username":
result.get("usernames", [""])[0] if
result.get("usernames") else "",
            "full_name": name,
            "first_name": name,
            "phone_info": {
                "success": True,
                "number":
result.get("number") or
record.get("phone"),
                "country":
result.get("country") or "-",
                "country_code":
result.get("country_code") or "-",
            },

```

```

        "is_bot": False,
        "is_verified": False,
        "is_premium": False,
        "is_scam": False,
        "is_fake": False,
        "is_restricted": False,
        "status": "offline",
        "dc_id": None,
        "common_chats_count":
0,
        "_extra": {
            "contact_links":
record.get("contact_links", []),
            "groups":
record.get("groups", []),

            "interested_users_count":
record.get("interested_users_count", 0),
            "interests":
record.get("interests", []),
        }
    }
    except Exception as e:
        logger.exception("tg2phone API
failed")
        return {"success": False,
"message": str(e)}

async def fetch_phone_info(number: str) ->
dict:
    timeout =
aiohttp.ClientTimeout(total=15, connect=5)
    async with aiohttp.ClientSession() as
s:

```

```
        async with s.get(PHONE_API_URL,
params={"number": number}, timeout=timeout)
as r:
            if r.status != 200:
                return {"success": False,
"message": f"API Error {r.status}"}
                return await
r.json(content_type=None)
```

```
#
```

```
# 🍷 FORMATTING
```

```
#
```

```
def hv(val, fallback="-", maxlen=300) ->
str:
    if val is None or
str(val).strip().lower() in ("", "null",
"none"):
        return fallback
    s = str(val).strip()
    return he(s[:maxlen] + "..." if len(s) >
maxlen else s)
```

```
STATUS_MAP = {
    "recently":      "🟡 Recently",
    "online":        "🟢 Online",
    "offline":       "🔴 Offline",
    "long_time_ago": "⬛ Long ago",
    "within_week":   "🟠 Within week",
    "within_month":  "🟡 Within month",
```

```

}
DC_MAP = {1:"DC1 🇺🇸 Miami", 2:"DC2 🇳🇱
Amsterdam",
          3:"DC3 🇺🇸 Miami", 4:"DC4 🇳🇱
Amsterdam", 5:"DC5 🇸🇬 Singapore"}
def bi(v): return "✅" if v else "❌"

def format_tg_result(d: dict) -> str:
    uname = d.get("username") or ""
    uid   = d.get("user_id") or "-"
    fname = d.get("full_name") or
d.get("first_name") or "-"
    bio   = d.get("bio")
    status =
STATUS_MAP.get(d.get("status",""),
hv(d.get("status","")))
    dc_id = d.get("dc_id")
    dc     = DC_MAP.get(dc_id, "-") if
dc_id else "-"
    cc     = d.get("common_chats_count") or
0
    ph     = d.get("phone_info") or {}
    phone_line = ""
    if isinstance(ph, dict) and
ph.get("success") and ph.get("number"):
        phone_line = (
            "\n📞 <b>Phone</b>\n"
            f"└─ Number   : <code>
{hv(ph.get('number'))}</code>\n"
            f"└─ Country :
{hv(ph.get('country'))}
{hv(ph.get('country_code',''))}\n"
        )
    flags = []

```

```

        if d.get("is_scam"):
            flags.append("🚫 Scam")
        if d.get("is_fake"):
            flags.append("⚠️ Fake")
        if d.get("is_restricted"):
            flags.append("🔒 Restricted")
        flags_line = "\n⚠️ " + " | "
    ".join(flags) + "\n" if flags else ""
    name_line = f"👤 <b>{hv(fname)}</b>"
    if uname:
        name_line += f" | @{he(uname)}"

    extra = d.get("_extra", {})
    extra_lines = ""
    if extra.get("contact_links"):
        extra_lines += "\n📁 <b>Other
Contacts</b>\n"
        for link in extra["contact_links"]
[:5]:
            extra_lines += f"  L
{he(link)}\n"
    if extra.get("groups"):
        extra_lines += "\n👥
<b>Groups</b>\n"
        for g in extra["groups"][:5]:
            extra_lines += f"  L
{he(g[:60])}...\n"
    if extra.get("interested_users_count"):
        extra_lines += f"\n👁️ Interested
Users: {extra['interested_users_count']}\n"
    if extra.get("interests"):
        extra_lines += f"📌 Interests: {' ,
'.join(he(i) for i in extra['interests']
[:5])}\n"

```

```

        return (
            f"{name_line}\n"
            f"🆔 <code>{uid}</code>\n"
            f"👁 Status : {status}\n"
            f"💻 DC : {dc}\n"
            + (f"💬 Common : {cc}\n" if cc else
            "")
            + (f"📝 Bio : <i>{hv(bio,
maxlen=150)}</i>\n" if bio else "")
            + "\n"
            + f"🤖 Bot :
{bi(d.get('is_bot'))}\n"
            + f"✅ Verified:
{bi(d.get('is_verified'))}\n"
            + f"★ Premium :
{bi(d.get('is_premium'))}\n"
            + flags_line
            + phone_line
            + extra_lines
            + f"\n💎 <b>Made by
@{OWNER_USERNAME}</b>"
        )

def format_phone_result(data: dict, number:
str) -> str:
    ICONS = {
        "name": "👤", "father_name": "👴",
        "address": "🏠", "alternate": "☎",
        "circle": "📶", "aadhar": "🇮🇳",
        "email": "✉", "mobile": "📱",
    }
    SKIP = {"mobile", "success", "cached",
"proxyused", "attempt",

```

```

        "credit", "developer",
        "status", "count", "search_time", "tag"}

    lines = f"📄 <b>{he(number)}</b>\n\n"

    results = []
    if data.get("success"):
        outer_result = data.get("result")
    or {}
        if outer_result.get("success"):
            inner_result =
outer_result.get("result") or {}
            results =
inner_result.get("results") or []

    if not results:
        lines += "❌ <b>No data
found</b>\n"
    else:
        seen = set()
        unique = []
        for r in results:
            key = (r.get("name", ""),
r.get("father_name", ""), r.get("address",
""))

            if key not in seen:
                seen.add(key)
                unique.append(r)

        for i, r in enumerate(unique):
            if i > 0:
                lines +=
"—————\n"
                for k, v in r.items():

```

```

        if k.lower() in SKIP:
            continue
        if not v or str(v).strip()
in ("", "null", "none", "0"):
            continue
            icon = ICONS.get(k.lower(),
".")
            label = k.replace("_", "
").title()
            lines += f"{icon} <b>
{label}</b> : {hv(str(v), maxlen=300)}\n"
            lines += "\n"

        lines += f"◆ <b>Made by
@{OWNER_USERNAME}</b>"
        return lines

```

```
#
```

```
#  KEYBOARDS
```

```
#
```

```

def two_button_kb(query: str):
    return InlineKeyboardMarkup([
        [InlineKeyboardButton("📞
Telegram", callback_data=f"search_tg:
{query}"),
        InlineKeyboardButton("📞 Number
Info", callback_data=f"search_ph:
{query}")],
    ])

```

```

def cancel_kb():
    return
    InlineKeyboardMarkup([[InlineKeyboardButton
    ("✖ Cancel", callback_data="cancel")]])

#
=====
=====
# 🔍 CORE LOOKUPS
#
=====
=====

async def perform_lookup(update: Update,
ctx: ContextTypes.DEFAULT_TYPE, query:
str):
    reply_fn =
update.effective_chat.send_message
    try:
        user_id = update.effective_user.id
        await
        upsert_user(update.effective_user)

        allowed, limit_msg = await
        check_and_increment_lookup(user_id)
        if not allowed:
            await safe_send(reply_fn,
            limit_msg)
        return

        msg = await reply_fn("🔍
<b>Searching Telegram...</b>",
        parse_mode=HTML)

```

```

        except Exception:
            logger.exception("perform_lookup:
FAILED before search started (query=%r)",
query)
            try:
                await reply_fn("✗ <b>Internal
error before search started.</b>",
parse_mode=HTML)
            except Exception:

logger.exception("perform_lookup: could not
even send the error message")
            return

        try:
            await asyncio.sleep(0.5)
            data = await
asyncio.wait_for(fetch_info(query),
timeout=20)

            if not data or data.get("success")
== False or "error" in data:
                err = data.get("message") or
data.get("error") or "Not found"
                await safe_edit(msg, f"✗
<b>Error:</b> <code>{he(str(err))}</code>")
                return

            await safe_edit(msg,
format_tg_result(data))

        except asyncio.TimeoutError:
            await safe_edit(msg, "✗
<b>Timeout!</b>")

```

```

        except aiohttp.ClientError:
            logger.exception("perform_lookup:
NETWORK ERROR")
            await safe_edit(msg, "❌ <b>Network
Error!</b>")
        except Exception as e:
            logger.exception("perform_lookup:
UNEXPECTED ERROR")
            await safe_edit(msg, f"❌ <b>Error:
</b> <code>{he(str(e))}</code>")

    async def perform_phone_lookup(update:
Update, ctx: ContextTypes.DEFAULT_TYPE,
number: str):
        reply_fn =
update.effective_chat.send_message
        try:
            user_id = update.effective_user.id
            await
            upsert_user(update.effective_user)

            allowed, limit_msg = await
            check_and_increment_lookup(user_id)
            if not allowed:
                await safe_send(reply_fn,
limit_msg)
            return

            clean_number =
            number.strip().replace(" ", "").replace("-",
            "")
            if not
            clean_number.lstrip("+").isdigit() or
            len(clean_number.lstrip("+")) < 7:

```

```

        await safe_send(reply_fn,
                        "❌ <b>Invalid
number!</b>\n"

                        "Example:
<code>9876543210</code> or
<code>+919876543210</code>")
        return
        number = clean_number

        msg = await reply_fn("📞
<b>Searching phone info...</b>",
        parse_mode=HTML)
        except Exception:

        logger.exception("perform_phone_lookup:
        FAILED before search started (number=%r)",
        number)
        try:
            await reply_fn("❌ <b>Internal
        error before search started.</b>",
        parse_mode=HTML)
        except Exception:

        logger.exception("perform_phone_lookup:
        could not even send the error message")
        return

        try:
            await asyncio.sleep(0.5)
            data = await
        asyncio.wait_for(fetch_phone_info(number),
        timeout=15)

        if not data or data.get("success")

```

```

== False or "error" in data:
    err = data.get("message") or
data.get("error") or "Not found"
    await safe_edit(msg, f"❌
<b>Error:</b> <code>{he(str(err))}</code>")
    return

    await safe_edit(msg,
format_phone_result(data, number))

    except asyncio.TimeoutError:
        await safe_edit(msg, "❌
<b>Timeout!</b>")
    except aiohttp.ClientError:

logger.exception("perform_phone_lookup:
NETWORK ERROR")
    await safe_edit(msg, "❌ <b>Network
Error!</b>")
    except Exception as e:

logger.exception("perform_phone_lookup:
UNEXPECTED ERROR")
    await safe_edit(msg, f"❌ <b>Error:
</b> <code>{he(str(e))}</code>")

#
=====
=====

# 📡 COMMANDS & HANDLERS
#
=====
=====

```

```

async def start(update: Update, ctx:
ContextTypes.DEFAULT_TYPE):
    await
    upsert_user(update.effective_user)
    fname =
he(update.effective_user.first_name or
"User")

    await safe_send(
        update.message.reply_text,
        f"🚬 <b>Smoke Bot</b>\n\n"
        f"👋 Welcome, <b>{fname}</b>!\n\n"
        f"Username, User ID ya Phone Number
bhejo – 2 options milenge:\n"
        f"📱 <b>Telegram Info</b> –
username/ID se details nikalo\n"
        f"☎️ <b>Number Info</b> – phone
number se info nikalo\n\n"
        f"📊 Daily limit: <code>
{DAILY_LIMIT}</code> lookups/day\n"
        f"\n💎 <b>Made by @{OWNER_USERNAME}
</b>",
    )

async def broadcast_cmd(update: Update,
ctx: ContextTypes.DEFAULT_TYPE):
    if update.effective_user.id !=
OWNER_ID:
        return
    ctx.user_data["waiting"] = "broadcast"
    await safe_send(
        update.message.reply_text,
        f"📢 <b>Broadcast</b>\n\nSabhi users
ko bhejni wali message type karo:",

```

```

        reply_markup=cancel_kb(),
    )

    async def smart_message(update: Update,
                             ctx: ContextTypes.DEFAULT_TYPE):
        try:
            await _smart_message_impl(update,
                                       ctx)
        except Exception:
            logger.exception("smart_message:
UNCAUGHT ERROR")
            try:
                await
update.effective_chat.send_message(
                    "❌ <b>Kuch galat ho gaya.
</b>", parse_mode=HTML,
                )
            except Exception:

logger.exception("smart_message: could not
even send the error message")

    async def _smart_message_impl(update:
Update, ctx: ContextTypes.DEFAULT_TYPE):
        text    = update.message.text.strip()
        uid     = update.effective_user.id
        waiting = ctx.user_data.get("waiting")
        await
upsert_user(update.effective_user)

        if waiting == "broadcast":
            if uid != OWNER_ID:
                return
            ctx.user_data.pop("waiting", None)

```

```

        users = await get_all_users()
        sent = failed = 0
        msg = await
update.message.reply_text("🔊
Broadcasting...")

        for u in users:
            try:
                await
ctx.bot.send_message(u["user_id"], text,
parse_mode=HTML)
                sent += 1
            except Exception:
                failed += 1

        await safe_edit(msg, f"✅
<b>Broadcast Done!</b>\n└ Sent    :
{sent}\n└ Failed : {failed}")
        return

    if ctx.user_data.pop("waiting_ph",
False):
        await perform_phone_lookup(update,
ctx, text)
        return

    is_phone    =
text.lstrip("+").isdigit() and
len(text.lstrip("+")) >= 7
    is_userid  = text.lstrip("-
").isdigit() and not is_phone
    is_username = text.startswith("@")

    if is_username or is_userid or

```

```

is_phone:
    await safe_send(
        update.message.reply_text,
        f"🔍 <b>Detected:</b> <code>
{he(text)}</code>\n\nSearch direction
choose karo:",

reply_markup=two_button_kb(text),
    )
else:
    await safe_send(
        update.message.reply_text,
        f"🚬 <b>Smoke Bot</b>\n\n"
        f"Kuch bhejo:\n"
        f"• <code>@username</code> –
Telegram username\n"
        f"• <code>123456789</code> –
Telegram User ID\n"
        f"• <code>9876543210</code> –
Phone number\n\n"
        f"♦ <b>Made by
@{OWNER_USERNAME}</b>",
    )

async def button_handler(update: Update,
ctx: ContextTypes.DEFAULT_TYPE):
    q = update.callback_query
    try:
        await _button_handler_impl(update,
ctx)
    except Exception:
        logger.exception("button_handler:
UNCAUGHT ERROR | data=%r", q.data if q else
None)

```

```

        try:
            await
            update.effective_chat.send_message(
                "❌ <b>Kuch galat ho gaya  

button dabane par.</b>", parse_mode=HTML,
            )
        except Exception:

            logger.exception("button_handler: could not  

even send the error message")

    async def _button_handler_impl(update:
Update, ctx: ContextTypes.DEFAULT_TYPE):
        q = update.callback_query
        await q.answer()
        data = q.data

        if data == "cancel":
            ctx.user_data.pop("waiting", None)
            try:
                await q.message.delete()
            except Exception:
                pass
            return

        if data.startswith("search_tg:"):
            query = data[len("search_tg:"):]
            try: await q.message.delete()
            except Exception: pass
            await perform_lookup(update, ctx,
query)
            return

        if data.startswith("search_ph:"):

```

```

        number = data[len("search_ph:")]
        try: await q.message.delete()
        except Exception: pass
        clean =
number.lstrip("+").replace(" ",
        "").replace("-", "")
        if clean.isdigit() and len(clean)
>= 7:
            await
perform_phone_lookup(update, ctx, number)
        else:
            ctx.user_data["waiting_ph"] =
True
            await safe_send(

update.effective_chat.send_message,
            "📞 <b>Number
Info</b>\n\nPhone number
bhejo:\n<i>Example: 9876543210</i>",
            reply_markup=cancel_kb(),
            )
        return

#
=====
=====

# 🚒 GLOBAL ERROR HANDLER
#
=====
=====

async def error_handler(update: object,
context: ContextTypes.DEFAULT_TYPE):
    logger.error("Unhandled exception while

```

```

processing update: %s", update,
exc_info=context.error)

#
=====

=====
# 🚀 MAIN
#
=====

=====

async def post_init(app):
    await init_db()

def main():
    app =
    Application.builder().token(BOT_TOKEN).post
_init(post_init).build()
    app.add_handler(CommandHandler("start",
start))

    app.add_handler(CommandHandler("broadcast",
broadcast_cmd))

    app.add_handler(CallbackQueryHandler(button
_handler))

    app.add_handler(MessageHandler(filters.TEXT
& ~filters.COMMAND, smart_message))
    app.add_error_handler(error_handler)
    logger.info("🚀 Smoke Bot Started -
Lean Mode")

app.run_polling(drop_pending_updates=True)

```

```
if __name__ == "__main__":  
    main()
```